

Лабораторная работа № 4

Тема работы: Библиотеки динамической компоновки DLL

Цель работы: Научиться использовать библиотеки динамической компоновки при разработке приложений

Теоретические сведения по изучению темы

Библиотеки динамической компоновки (dynamic link libraries – DLL) являются исполняемыми файлами особого формата, которые содержат функции, данные или ресурсы, доступные для других приложений.

Особый формат модулей DLL предполагает наличие в них разделов импорта и экспорта. Раздел экспорта указывает те идентификаторы объектов (функций, классов, переменных), доступ к которым разрешен для клиентов.

подавляющее большинство DLL (за исключением DLL, содержащих только ресурсы) импортирует функции из системных DLL - kernel32.dll, user32.dll, gdi32.dll и других библиотек.

Применение DLL может дать ряд преимуществ:

- расширение функциональности приложения. DLL можно загружать в адресное пространство процесса на этапе выполнения, что позволит программе, определив, какие действия от нее требуются, подгружать нужный код;
- более простое управление проектом. Использование DLL упрощает отладку, тестирование и сопровождение проекта;
- экономия памяти. Если одну и ту же DLL используют несколько приложений, то в оперативной памяти хранится только один ее экземпляр, доступный этим приложениям;
- разделение ресурсов. DLL могут содержать такие ресурсы, как строки, растровые изображения, шаблоны диалоговых окон. Этими ресурсами может воспользоваться любое приложение;
- возможность использования разных языков программирования.

Исполняемый код в DLL не предполагает автономного использования. Содержимое каждого DLL-файла загружается приложением и проецируется на адресное пространство вызывающего процесса. Это достигается либо за счет неявного связывания при загрузке, либо за счет явного связывания в период выполнения.

Важно понимать, что процесс, загрузивший DLL, получает собственную копию глобальных данных, используемых этой библиотекой. Это защищает DLL от ошибок приложений, а процессы, использующие DLL, от взаимного влияния друг на друга.

Вызов функций из DLL

Существует три способа загрузки DLL:

- неявная;
- явная;
- отложенная.

Рассмотрим неявную загрузку DLL. Для построения приложения, рассчитанного на неявную загрузку DLL, необходимо иметь:

- библиотечный включаемый файл с описаниями используемых объектов из DLL (прототипы функций, объявления классов и типов). Этот файл используется компилятором;
- LIB-файл со списком импортируемых идентификаторов. Этот файл нужно добавить в настройки проекта (в список библиотек, используемых компоновщиком).

Компиляция проекта осуществляется обычным образом. Используя объектные модули и LIB-файл, а также учитывая ссылки на импортируемые идентификаторы, компоновщик (линкер, редактор связей) формирует загрузочный EXE-модуль. В этом модуле компоновщик помещает также раздел импорта, где перечисляются имена всех необходимых DLL-модулей. Для каждой DLL в разделе импорта указывается, на какие имена функций и переменных встречаются ссылки в коде исполняемого файла. Эти сведения будет использовать загрузчик операционной системы.

Что же происходит на этапе выполнения клиентского приложения? После запуска EXE-модуля загрузчик операционной системы выполняет следующие операции:

- создает виртуальное адресное пространство для нового процесса и проецирует на него исполняемый модуль;
- анализирует раздел импорта, определяя все необходимые DLL-модули и тоже проецируя их на адресное пространство процесса.

DLL может импортировать функции и переменные из другой DLL. А значит, у нее может быть собственный раздел импорта, для которого необходимо повторить те же действия. В результате на инициализацию процесса может уйти довольно длительное время.

После отображения EXE-модуля и всех DLL-модулей на адресное пространство процесса его первичный поток готов к выполнению, и приложение начинает работу.

Недостатками неявной загрузки являются обязательная загрузка библиотеки, даже если обращение к ее функциям не будет происходить, и, соответственно, обязательное требование к наличию библиотеки при компоновке.

Явная загрузка DLL устраняет отмеченные выше недостатки за счет некоторого усложнения кода. Программисту приходится самому заботиться о загрузке DLL и подключении экспортируемых функций. Зато явная загрузка позволяет подгружать DLL по мере необходимости и дает возможность программе обрабатывать ситуации, возникающие при отсутствии DLL.

В случае явной загрузки процесс работы с DLL происходит в три этапа:

- загрузка DLL с помощью функции LoadLibrary (или ее расширенного аналога LoadLibraryEx). В случае успешной загрузки функция возвращает дескриптор hLib типа HMODULE, что позволяет в дальнейшем обращаться к этой DLL;

- вызовы функции GetProcAddress для получения указателей на требуемые функции или другие объекты. В качестве первого параметра функция GetProcAddress получает дескриптор hLib, в качестве второго параметра -адрес строки с именем импортируемой функции. Далее полученный указатель используется клиентом. Например, если это указатель на функцию, то осуществляется вызов нужной функции;

- когда загруженная динамическая библиотека больше не нужна, рекомендуется ее освободить, вызвав функцию FreeLibrary. Освобождение библиотеки не означает, что операционная система немедленно удалит ее из памяти. Задержка выгрузки предусмотрена на тот случай, когда эта же DLL через некоторое время вновь понадобится какому-то процессу. Но если возникнут проблемы с оперативной памятью, Windows в первую очередь удаляет из памяти освобожденные библиотеки.

Рассмотрим отложенную загрузку DLL. DLL отложенной загрузки (delay-load DLL) – это неявно связываемая DLL, которая не загружается до тех пор, пока код не обратится к какому-нибудь экспортируемому из нее идентификатору. Такие DLL могут быть полезны в следующих ситуациях:

– если приложение использует несколько DLL, его инициализация может занимать длительное время, требуемое загрузчику для проецирования всех DLL на адресное пространство процесса. DLL отложенной загрузки позволяют решить эту проблему, распределяя загрузку DLL в ходе выполнения приложения;

– если приложение предназначено для работы в различных версиях ОС, то часть функций может появиться лишь в поздних версиях ОС и не использоваться в текущей версии. Но если программа не вызывает конкретной функции, то DLL ей не нужна, и она может спокойно продолжать работу. При обращении же к несуществующей функции можно предусмотреть выдачу пользователю соответствующего предупреждения.

Для реализации метода отложенной загрузки требуется дополнительно в список библиотек компоновщика добавить не только нужную библиотеку импорта MyLib.lib, но еще и системную библиотеку импорта delayimp.lib. Кроме этого, требуется добавить в опциях компоновщика флаг /delayload:MyLib.dll.

Перечисленные настройки заставляют компоновщик выполнить следующие операции:

- внедрить в EXE-модуль специальную функцию delayLoadHelper;
- удалить MyLib.dll из раздела импорта исполняемого модуля, чтобы загрузчик операционной системы не пытался выполнить неявную загрузку этой библиотеки на этапе загрузки приложения;
- добавить в EXE-файл новый раздел отложенного импорта со списком функций, импортируемых из MyLib.dll;
- преобразовать вызовы функций из DLL к вызовам delayLoadhelper.

На этапе выполнения приложения вызов функции из DLL реализуется обращением к delayLoadHelper. Эта функция, используя информацию из раздела отложенного импорта, вызывает сначала LoadLibrary, а затем GetProcAddress. Получив адрес DLL-функции, delayLoadHelper делает так, чтобы в дальнейшем эта DLL-функция вызывалась напрямую. Отметим, что каждая функция в DLL настраивается индивидуально при первом ее вызове.

Функция входа/выхода DLL. Для решения указанных проблем вы можете включить в состав DLL специальную функцию точки входаDllMain. Эта функция вызывается операционной системой в следующих случаях:

- когда DLL проецируется на адресное пространство процесса (подключение DLL); когда процессом, загрузившим DLL, вызывается новый поток;
- когда завершается поток, принадлежащий процессу, который связан с DLL;
- когда процесс освобождает DLL (отключение DLL).

В момент вызова функция получает информацию от операционной системы через свои параметры.

Первый параметр, `hinstDLL`, принимает значение дескриптора модуля DLL, являющееся, по сути, виртуальным адресом загрузки DLL. Если в библиотеке имеются вызовы функций, которым нужен данный дескриптор, необходимо сохранить значение `hinstDLL` в глобальной переменной.

Второй параметр, `fdwReason`, может принимать одно из следующих значений:

- `DLLPROCESSATTACH` – уведомление о том, что DLL загружена в адресное пространство процесса либо в результате его старта, либо в результате вызова функции `LoadLibrary`;
- `DLLTHREADATTACH` – уведомление о том, что текущий процесс создал новый поток. Это уведомление посылается всем DLL, подключенным к процессу. Вызов `DllMain` происходит в контексте нового потока;
- `DLLTHREADDETACH` – уведомление о том, что поток корректно завершается. Вызов `DllMain` происходит в контексте завершающегося потока;
- `DLLPROCESSDETACH` – уведомление о том, что DLL отключается от адресного пространства процесса в результате одного из трех событий: а) неудачное завершение загрузки DLL; б) вызов функции `FreeLibrary`; в) завершение процесса.

Если при вызове функции используется первый параметр со значением `DLLPROCESSATTACH`, то по значению третьего параметра можно выяснить, каким способом загружается DLL. При явной загрузке параметр `lReserved` равен нулю, а при неявной загрузке принимает ненулевое значение.

Следует отметить следующие моменты работы с функцией входа-выхода DLL:

1. Поток, вызвавший `DllMain` со значением `DLLPROCESSATTACH`, не вызывает повторно `DllMain` со значением `DLLTHREADATTACH`.

2. Когда DLL загружается вызовом функции LoadLibrary, существующие потоки не вызывают DllMain для вновь загруженной библиотеки.

3. Функция DllMain не вызывается, если поток или процесс завершаются по причине вызова функции TerminateThread или TerminateProcess.

Поскольку функция DllMain должна обрабатывать все возможные причины своего вызова, то в ее коде обратно анализируется dwReason и выполняются необходимые действия по инициализации или освобождению ресурсов.

Порядок выполнения:

При создании нового Win32-проекта выбрать тип DLL. Будет создан файл реализации dlmain.cpp.

При разработке библиотеки, содержащей ресурсы, необходимо их добавить в библиотеку динамической компоновки (рисунок 1). DLL-библиотека должна быть создана с использованием def-файла: EXPORTS hIcon.

```
#include "stdafx.h"
#include "resource.h"
__declspec(dllexport) HICON hIcon;
BOOL APIENTRY DllMain( HMODULE hModule, // дескриптор библиотеки, прис
                      DWORD ul_reason_for_call, // код уведомления
                      LPVOID lpReserved // зарезервировано
                      )
{
    switch (ul_reason_for_call)
    {
        case DLL_PROCESS_ATTACH:
            hIcon = LoadIcon(hModule, MAKEINTRESOURCE(IDI_ICON1));
            break;
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            DestroyIcon(hIcon);
            break;
    }
    return TRUE;
}
```

Рисунок 1 - Реализация функции DllMain

Библиотеку нужно скомпилировать и убедиться, что папка проекта содержит lib- и dll-файлы. Для явной загрузки библиотеки в созданный проект добавим dll-файл созданной библиотеки. В сообщении WM_CREATE необходимо получить дескриптор библиотеки: hDll = LoadLibrary(_T("DllIcon")), передавая ей в качестве параметра имя DLL-файла. Далее, функцией GetProcAddress() находим дескриптор иконки, уже загруженной в библиотеке, передавая ей дескриптор иконки как текстовую

строку: `hIcon = *((HICON*)GetProcAddress(hDll, "hIcon"))`. Переменная `hIcon` описана как `HICON`.

После этого можем изменить малую иконку класса окна `SetClassLong(hWnd, GCL_HICONSM, (LONG)hIcon)`.

Для неявной загрузки библиотеки динамической компоновки. В DLL определяем функции (рисунок 2).

```
// dllmain.cpp: определяет точку входа для приложения DLL.
#include "stdafx.h"
__declspec(dllexport) BOOL WINAPI Sum(int *, int);
BOOL APIENTRY DllMain( HMODULE hModule,
                      DWORD ul_reason_for_call,
                      LPVOID lpReserved
                      )
{
    switch (ul_reason_for_call)
    {
        case DLL_PROCESS_ATTACH:
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}

BOOL WINAPI Sum(int *a, int n)
{
    int sum=0;
    for (int i = 0; i < n; i++)
    {
        sum += a[i];
    }
    return (sum);
}
```

Рисунок 2 - Реализация функции `DllMain`

Библиотеку нужно скомпилировать и убедиться, что папка проекта содержит `lib`- и `dll`-файлы. К проекту, подключаем `lib`-файл динамически подключаемой библиотеки (рисунок 3).

Объявляем прототип функции: `WINGDIAPI BOOL WINAPI Sum(int*, int)`.

И вызываем данную функцию, согласно алгоритма программы `int k=Sum(mas, 10)`.

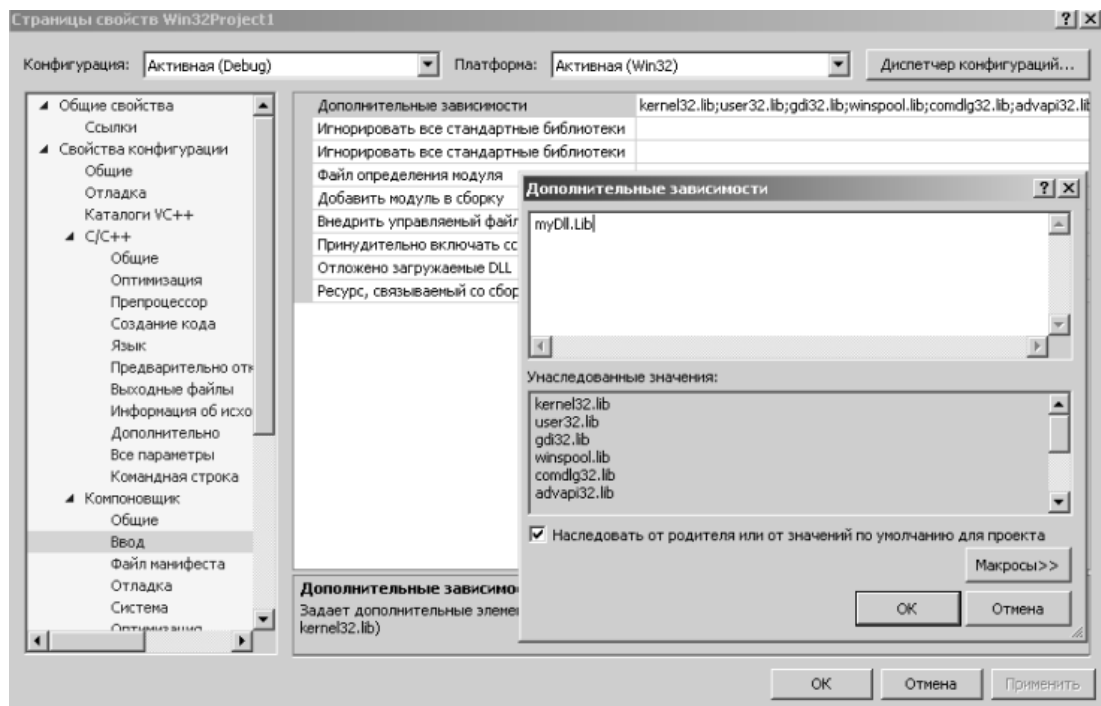


Рисунок 3 – Подключение файла lib к проекту

Задание:

Создайте две библиотеки. Первая библиотека должна быть явно подключена к вашему приложению, а вторая — неявно. Обе библиотеки должны содержать ресурсы: иконку, курсор, а также функции вычисления суммы(*dll_1*) и разности(*dll_2*) чисел. В приложении используйте ресурсы и функции из обеих библиотек: отобразите иконку в заголовке окна, курсор в приложении и вывод результата функций в клиентской области по нажатию на кнопку.

Дополнительное задание (на практике изучить процесс подмены DLL, понять риски подмены DLL и влияние на выполнение программ)

Создайте третью DLL, в которой реализуйте такую же функцию, как и в первой DLL, которую вы загружали явно. В этой функции замените оригинальную логику функции вычисления суммы чисел на вызов MessageBox, который будет отображать сообщение "DLL hijacking :)".

1. Реализация:

- Создайте новый проект для третьей DLL.
- Определите функцию с тем же именем, что и в первой DLL.
- Внутри функции реализуйте вызов MessageBox.

2. Тестирование:

- Убедитесь, что ваша третья DLL загружается вместо первой.
- Проверьте, что при вызове функции отображается сообщение.